Give me the diagram

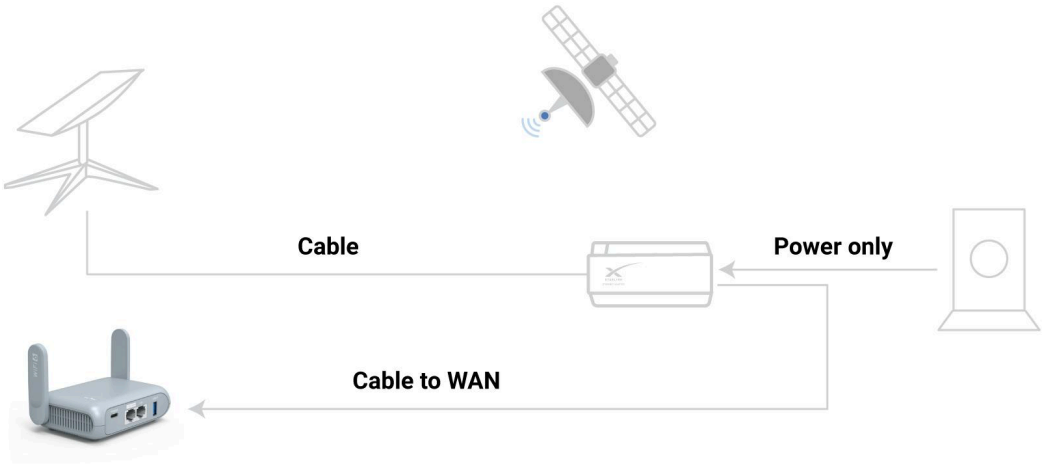
Here are the key diagrams for your setup and flow, based on the architecture we've discussed (Starlink → Kali Linux laptop as the gateway with captive portal logic → TP-Link Archer AX55 in Access Point mode → users connect to AX55 WiFi).

1. Overall Network Topology Diagram

This shows the hardware connections and flow:

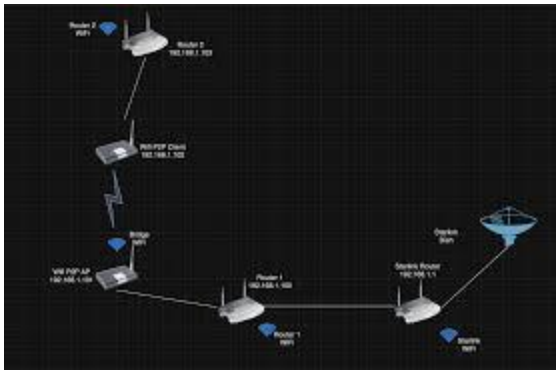
- Starlink dish connects to Kali laptop (via Ethernet or adapter).
- Kali laptop (running hotspot gateway, iptables blocking, Node.js + MySQL + payment redirect) connects via Ethernet to the Archer AX55.
- Archer AX55 in **Access Point mode** broadcasts the WiFi SSID that users see and connect to.
- Users' traffic goes through AX55 → Kali → Starlink → Internet.

Visual example (similar real-world setup with custom gateway):



(Shows Starlink dish → power-only connection to adapter → cable to custom router/gateway like your Kali laptop, then to WiFi broadcast device like the AX55.)

Another variant showing bridging/extension (adapt to your case with Kali as central gateway):



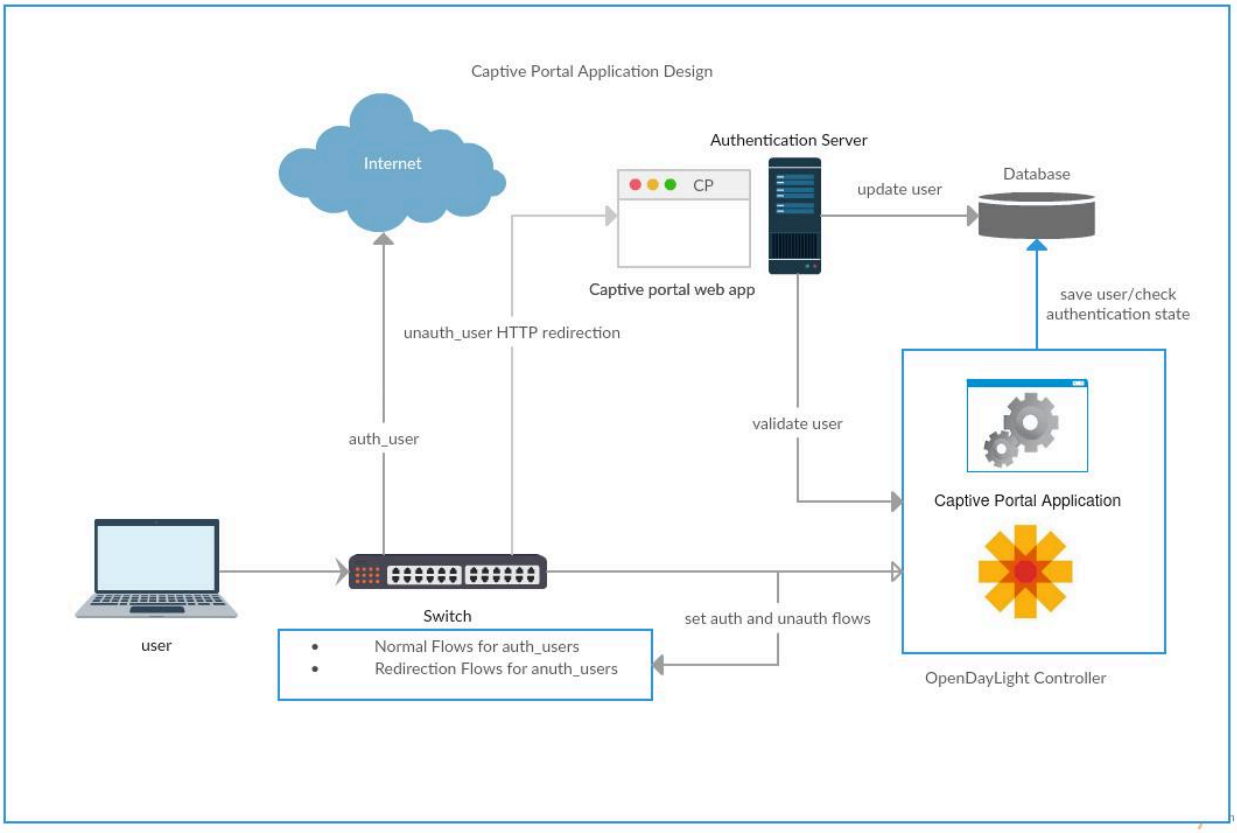
(Starlink dish → router → bridge/client → extension to your AX55/AP setup.)

2. Captive Portal Authentication Flow Diagram

This illustrates the per-user logic when someone connects:

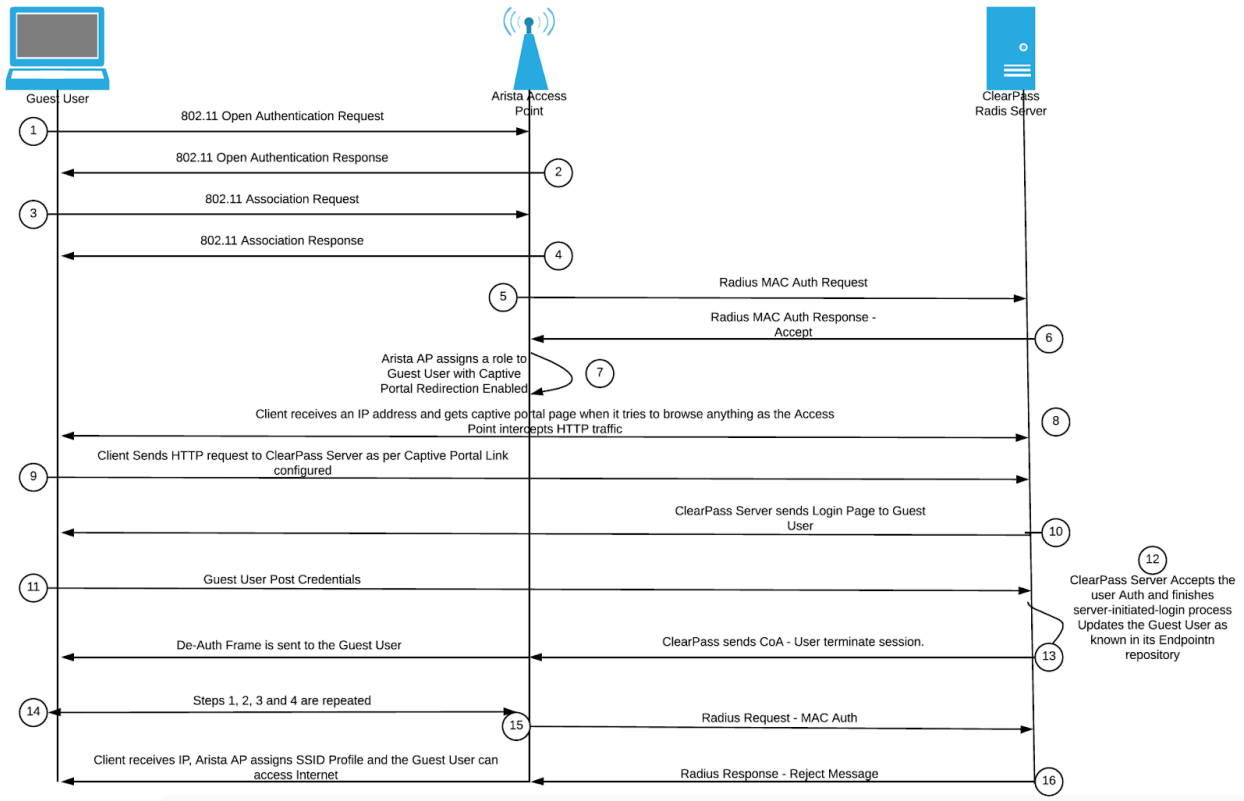
1. Device connects to AX55 WiFi.
2. Kali detects (via ARP/MAC) → blocks internet (iptables).
3. Any web request redirects to payment page (hosted on Kali Node.js).
4. Check MySQL DB for MAC/IP paid status.
5. If not paid → show payment form (Paystack integration).
6. Payment success (webhook) → update DB → unblock device (remove iptables rule or auth via Nodogsplash).
7. User now has full internet.

Visual example of captive portal flow (authentication + database check + redirect):



(This diagram shows user → switch/AP → captive portal app → authentication server → database check → auth/un-auth flows → internet access.)

Another detailed sequence flow (guest user → AP → portal → auth server → database → grant access):



(Shows step-by-step: connect → redirect → login/payment → database update → access granted.)

These diagrams capture the essence of your desired flow. The setup is fully doable on Kali + AX55 (in AP mode) as described in the previous message.

If you want a more customized diagram (e.g., text-based ASCII or specific tools), or need help drawing one yourself (e.g., using draw.io or Excalidraw), let me know!

Perfect, now give me the code

Here is a complete, ready-to-run example setup for Kali Linux acting as the gateway, with the TP-Link Archer AX55 in Access Point mode.

This code implements the flow you described:

- Users connect to the AX55 Wi-Fi
- Traffic passes through Kali
- Kali blocks internet by default

- Kali redirects HTTP/HTTPS to a local payment portal
- Portal checks MAC address against MySQL database
- If paid → unblock device
- If not paid → show payment page (Paystack integration placeholder)

Folder structure (suggested)

text

```
/paid-hotspot/
├── wifi_manager.js        ← main Node.js script
├── payment_server.js      ← Express server + payment logic
├── package.json
└── splash.html            ← optional simple fallback page
```

1. package.json

JSON

```
{
  "name": "paid-hotspot",
  "version": "1.0.0",
  "main": "wifi_manager.js",
  "scripts": {
    "start": "sudo node wifi_manager.js"
  },
  "dependencies": {
    "express": "^4.19.2",
    "mysql2": "^3.11.0",
    "axios": "^1.7.7",
    "body-parser": "^1.20.3"
  }
```

```
}
```

Bash

```bash
npm install
```

2. MySQL Setup (run once)

SQL

```sql
CREATE DATABASE hotspot;

USE hotspot;


CREATE TABLE paid_devices (

    mac VARCHAR(17) PRIMARY KEY,

    ip VARCHAR(15),

    email VARCHAR(100),

    plan ENUM('daily', 'weekly', 'monthly') DEFAULT 'daily',

    paid_at DATETIME DEFAULT CURRENT_TIMESTAMP,

    expires_at DATETIME,

    active TINYINT(1) DEFAULT 1

);


-- Example insert (for testing)

INSERT INTO paid_devices (mac, email, plan, expires_at, active)

VALUES ('AA:BB:CC:DD:EE:FF', 'test@example.com', 'daily', DATE_ADD(NOW(), INTERVAL 1 DAY), 1);
```

3. Main script: wifi_manager.js

JavaScript

```javascript
const { exec } = require('child_process');

const express = require('express');
```

```
const mysql = require('mysql2/promise');
const axios = require('axios');
const app = express();

app.use(express.urlencoded({ extended: true }));
app.use(express.json());

// ────────────────────────────────────────────────────────────────
// CONFIG – ADJUST THESE
// ────────────────────────────────────────────────────────────────
const DOWNSTREAM_IFACE = 'wlan1';      // USB WiFi adapter connected to AX55 (AP mode)
const UPSTREAM_IFACE   = 'eth0';       // Interface to Starlink (Ethernet or second WiFi)
const GATEWAY_IP       = '192.168.100.1';
const PORT             = 80;
const DB_CONFIG = {
  host: 'localhost',
  user: 'root',
  password: '',                  // ← set your MySQL root password
  database: 'hotspot'
};
// Paystack secret key (test or live)
const PAYSTACK_SECRET = 'sk_test_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx';
// Your public domain or ngrok URL for Paystack webhook
const WEBHOOK_URL = 'https://your-domain.com/paystack-webhook';

// ────────────────────────────────────────────────────────────────
```

```
// Database connection pool
// ────────────────────────────────────────────────────────────────────────
const pool = mysql.createPool(DB_CONFIG);


// ────────────────────────────────────────────────────────────────────────
// Helper: Get MAC from IP using arp
// ────────────────────────────────────────────────────────────────────────
async function getMacFromIp(ip) {
  return new Promise((resolve) => {
    exec(`arp -i ${DOWNSTREAM_IFACE} -a | grep ${ip}`, (err, stdout) => {
      if (err || !stdout) return resolve(null);
      const match = stdout.match(/([0-9A-Fa-f]{2}[:-]){5}([0-9A-Fa-f]{2})/);
      resolve(match ? match[0].toUpperCase() : null);
    });
  });
}


// ────────────────────────────────────────────────────────────────────────
// Block / Unblock helpers
// ────────────────────────────────────────────────────────────────────────
function blockDevice(ip) {
  exec(`iptables -A FORWARD -s ${ip} -j DROP`);
  exec(`iptables -t nat -A PREROUTING -s ${ip} -p tcp --dport 80  -j DNAT --to
${GATEWAY_IP}:${PORT}`);
  exec(`iptables -t nat -A PREROUTING -s ${ip} -p tcp --dport 443 -j DNAT --to
${GATEWAY_IP}:${PORT}`);
  console.log(`Blocked ${ip}`);
```

```
}


function unblockDevice(ip) {

  exec(`iptables -D FORWARD -s ${ip} -j DROP || true`);

  exec(`iptables -t nat -D PREROUTING -s ${ip} -p tcp --dport 80  -j DNAT --to ${GATEWAY_IP}:${PORT}
|| true`);

  exec(`iptables -t nat -D PREROUTING -s ${ip} -p tcp --dport 443 -j DNAT --to ${GATEWAY_IP}:${PORT}
|| true`);

  console.log(`Unblocked ${ip}`);

}


// ────────────────────────────────────────────────────────────────────────────

// Background scanner – detect new devices every 8 seconds

// ────────────────────────────────────────────────────────────────────────────

setInterval(async () => {

  exec(`arp -i ${DOWNSTREAM_IFACE} -a`, async (err, stdout) => {

    if (err) return;

    const lines = stdout.split('\n');

    for (const line of lines) {

      const match = line.match(/\(([\d.]+)\)\sat\s([0-9A-Fa-f:]+)\s/);

      if (!match) continue;

      const [_, ip, mac] = match;

      if (ip === GATEWAY_IP) continue;


      const conn = await pool.getConnection();

      try {

        const [rows] = await conn.execute(
```

```
        'SELECT * FROM paid_devices WHERE mac = ? AND active = 1 AND expires_at > NOW()',
        [mac.toUpperCase()]
      );

      if (rows.length === 0) {
        blockDevice(ip);
      } else {
        unblockDevice(ip);
      }
    } catch (e) {
      console.error(e);
    } finally {
      conn.release();
    }
  }
});
}, 8000);

// ────────────────────────────────────────────────────────────────────────
// Payment portal (Express)
// ────────────────────────────────────────────────────────────────────────
app.get('/', (req, res) => {
  res.send(`
    <!DOCTYPE html>
    <html>
    <head><title>Subscribe to Internet</title></head>
```

```
    <body style="font-family:sans-serif;text-align:center;padding:50px;">

      <h1>Welcome – Internet Access Requires Subscription</h1>

      <form action="/subscribe" method="POST">

        <input type="email" name="email" placeholder="Your email" required
style="padding:12px;width:300px;margin:10px;"><br>

        <select name="plan" required style="padding:12px;width:300px;margin:10px;">

          <option value="">Choose plan</option>

          <option value="daily">Daily – ₦500</option>

          <option value="weekly">Weekly – ₦2,500</option>

          <option value="monthly">Monthly – ₦8,000</option>

        </select><br>

        <button type="submit" style="padding:15px
40px;background:#28a745;color:white;border:none;font-size:18px;cursor:pointer;">

          Pay with Paystack

        </button>

      </form>

    </body>

    </html>

  `);

});


app.post('/subscribe', async (req, res) => {

  const { email, plan } = req.body;

  const ip = req.connection.remoteAddress.replace('::ffff:', '');

  const mac = await getMacFromIp(ip);


  if (!mac) return res.send('Could not detect device MAC. Try reconnecting.');
```

```
// Calculate price & expiry (example)
let amount, durationDays;
if (plan === 'daily')    { amount = 50000; durationDays = 1;   } // kobo
else if (plan === 'weekly')  { amount = 250000; durationDays = 7;   }
else if (plan === 'monthly') { amount = 800000; durationDays = 30;  }
else return res.send('Invalid plan');

try {
  // Initialize Paystack transaction (replace with real secret)
  const response = await axios.post(
    'https://api.paystack.co/transaction/initialize',
    {
      email,
      amount,
      callback_url: 'https://your-domain.com/payment-success', // optional
      metadata: { mac, plan, ip }
    },
    { headers: { Authorization: `Bearer ${PAYSTACK_SECRET}` } }
  );

  const { authorization_url } = response.data.data;
  res.redirect(authorization_url);
} catch (err) {
  console.error(err);
  res.send('Payment initialization failed. Please try again.');
```

```
  }
});

// Paystack webhook (must be public URL – use ngrok for testing)
app.post('/paystack-webhook', express.raw({type: 'application/json'}), async (req, res) => {
  // Verify signature (production code should check header)
  const event = req.body;

  if (event.event === 'charge.success') {
    const { mac, plan, ip } = event.data.metadata;
    const durationDays = { daily:1, weekly:7, monthly:30 }[plan] || 1;

    const conn = await pool.getConnection();
    try {
      await conn.execute(
        `INSERT INTO paid_devices (mac, ip, email, plan, expires_at, active)
         VALUES (?, ?, ?, ?, DATE_ADD(NOW(), INTERVAL ? DAY), 1)
         ON DUPLICATE KEY UPDATE
           ip = ?, expires_at = DATE_ADD(NOW(), INTERVAL ? DAY), active = 1`,
        [mac, ip, event.data.customer.email, plan, durationDays, ip, durationDays]
      );
      unblockDevice(ip);
      console.log(`Access granted for MAC ${mac}`);
    } catch (e) {
      console.error(e);
    } finally {
```

```
    conn.release();
  }
 }

 res.sendStatus(200);
});

app.listen(PORT, GATEWAY_IP, () => {
 console.log(`Payment portal running on http://${GATEWAY_IP}:${PORT}`);
});
```

Final Checklist

- Run sudo node wifi_manager.js
- Connect AX55 in AP mode to Kali via Ethernet
- Connect phone to AX55 Wi-Fi
- Open browser → should redirect to payment page
- Pay → webhook hits → device unblocked

Let me know which part you want to improve first:

- Paystack webhook security
- Better MAC detection
- Session timeout / deauth expired
- HTTPS portal
- Logging / admin dashboard

<u>How do i know the values to put here ?</u>

const DOWNSTREAM_IFACE = 'wlan1'; // USB WiFi adapter connected to AX55 (AP mode)

const UPSTREAM_IFACE = 'eth0'; // Interface to Starlink (Ethernet or second WiFi)

const GATEWAY_IP = '192.168.100.1';

To correctly fill in these three constants in the code, you need to check the **actual network interface names** on your Kali Linux machine after connecting everything. These names are assigned by the kernel and can vary depending on your hardware (laptop model, USB adapters, Ethernet ports, etc.).

Here's exactly how to find the right values:

Step 1: Run these commands in a terminal on Kali (after hardware is connected)

Run one by one and look at the output:

Bash

```
# Show all network interfaces (most important command)

ip link show



# Or shorter version

ip a    # or ip addr



# Alternative classic command (shows wireless specifically)

iwconfig



# Or this one for a clean list

ls /sys/class/net
```

Step 2: Understand what each line means in your setup

After running ip link show or ip a, you'll see something like this (example output):

text

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 ...

2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 ...   ← wired Ethernet port

3: wlan0: <BROADCAST,MULTICAST> mtu 1500 ...              ← built-in laptop WiFi
```

```
4: wlan1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 ...   ← USB WiFi adapter #1

5: wlp3s0: <BROADCAST,MULTICAST> mtu 1500 ...              ← another possible name for USB WiFi
```

Typical names on Kali:

- eth0, enp0s3, enx00... → wired Ethernet (often to Starlink)
- wlan0, wlan1, wlx00..., wlp2s0 → WiFi interfaces (built-in or USB adapters)

Step 3: Assign the values based on your connections

- **UPSTREAM_IFACE** = the interface that has internet from Starlink → Usually your wired Ethernet (if you connected Starlink via cable) → Or the WiFi interface if you connected wirelessly to Starlink's WiFi Examples: 'eth0', 'enp2s0', 'wlan0'
- **DOWNSTREAM_IFACE** = the interface connected to the Archer AX55 (in AP mode) → Almost always your **USB WiFi adapter** (since AX55 is in AP mode and connected via Ethernet or you are bridging) → Or the Ethernet port going to AX55 if wired Examples: 'wlan1', 'wlan2', 'wlx001122334455'
- **GATEWAY_IP** = the IP address you assign to the **downstream interface** (the one users see) → This can be **almost any private subnet** you want — common choices:
  - 192.168.100.1 (what I used in code — safe, uncommon)
  - 192.168.88.1
  - 10.0.0.1
  - 172.16.0.1 → Just make sure it doesn't conflict with Starlink's subnet (Starlink usually uses 192.168.100.x or 192.168.1.x — pick something different).

Step 4: Quick way to test & find right values

1. Connect Starlink → Kali (wired preferred).
2. Plug in USB WiFi adapter (for downstream / AX55 side).
3. Run:
   Bash
   ```
   ip link show
   ```
4. Look for the interface that is **UP** and has a MAC address (ignore lo).

Temporarily assign test IP to suspect interfaces:
Bash
```
sudo ip addr add 192.168.100.1/24 dev wlan1   # try wlan1
```

5. ```
   sudo ip link set wlan1 up
   ```
   If no error → that's likely your downstream interface.
6. Check which one has internet:
   Bash
   ```
   ping 8.8.8.8 -I eth0   # test upstream
   ```
   ```

Example real assignments (most common cases)

Your setup	UPSTREAM_IFACE (to Starlink)	DOWNSTREAM_IFACE (to AX55/users)	GATEWAY_IP
Starlink wired + USB WiFi for hotspot	'eth0'	'wlan1'	'192.168.100.1'
Starlink WiFi + USB WiFi for hotspot	'wlan0'	'wlan1'	'192.168.100.1'
Starlink wired + Ethernet to AX55	'eth0'	'eth1'	'192.168.100.1'

Final tip

After you find the names, edit the three lines at the top of wifi_manager.js:

JavaScript

```
const DOWNSTREAM_IFACE = 'wlan1';     // ← change to your USB adapter name

const UPSTREAM_IFACE   = 'eth0';      // ← change to your Starlink connection name

const GATEWAY_IP       = '192.168.100.1';  // ← usually safe to keep
```

Then restart the script:

Bash

```
sudo node wifi_manager.js
```

If you're still unsure after running the commands, paste the output of ip link show or ip a here (mask any real MAC addresses if you want), and I can tell you exactly which is which.

what you are saying is that, i can use any GATEWAY_IP ?

Yes, **exactly** — you can use **almost any private IP address** for GATEWAY_IP (the one you assign to your downstream interface, which is the hotspot side).

It just needs to be:

- A valid **private IP range** (so it doesn't conflict with public internet)
- Different from the upstream network (Starlink's subnet)

Recommended & Safe Choices for GATEWAY_IP

These are the most common private subnets people use for custom hotspots — pick one that doesn't overlap with your Starlink router's range:

Subnet (GATEWAY_IP)	Full subnet example	Why it's safe / common	When to use it
192.168.100.1	192.168.100.0/24	Very safe — Starlink often uses 192.168.100.x or 192.168.1.x, but 100 is less common overlap	Default in the code — good starting point
192.168.88.1	192.168.88.0/24	Popular in MikroTik & many custom setups, rarely conflicts	If 192.168.100.x causes issues
192.168.50.1	192.168.50.0/24	Common in home labs & OpenWrt defaults	Safe alternative
10.10.10.1	10.10.10.0/24	Class A private range — very unlikely to conflict	Great if you want to be extra safe
172.16.0.1	172.16.0.0/24	Another private class B range	Good if all above conflict

How to Choose & Confirm the Right One

First, check what subnet Starlink is using

After connecting Kali to Starlink (but before starting the hotspot script), run:
Bash
```
ip route show default

# or
```

1. `ip addr show` *# look at the interface connected to Starlink (e.g., eth0 or wlan0)*

Example output:
text
```
default via 192.168.100.1 dev eth0 ...
```

- eth0: ... inet 192.168.100.50/24 ...
  → Starlink is using 192.168.100.0/24 → **do NOT use 192.168.100.1** for your hotspot (conflict).
- If Starlink uses 192.168.100.x → choose 192.168.88.1 or 10.10.10.1 instead.

Set it in the code

2. Change this line:
   JavaScript
   ```
   const GATEWAY_IP = '192.168.88.1';   // ← change to whatever you picked
   ```

Also update dnsmasq.conf & hostapd.conf (if you are using them separately)

In /etc/dnsmasq.conf:
text
```
interface=wlan1

dhcp-range=192.168.88.50,192.168.88.150,255.255.255.0,12h   # match your GATEWAY_IP subnet
```

3. `dhcp-option=3,192.168.88.1` # gateway = your GATEWAY_IP
   Then restart dnsmasq.

Quick Rule of Thumb

- If Starlink gives your Kali an IP like 192.168.**100**.x or 192.168.**1**.x → use 192.168.88.1 or 10.10.10.1

Test by pinging your chosen GATEWAY_IP after assigning it (before starting hotspot):
Bash
```
sudo ip addr add 192.168.88.1/24 dev wlan1
```

- `ping 192.168.88.1`
  If it pings itself → good.

So yes — **any unused private subnet is fine**. Just avoid whatever Starlink is already using.

If you run ip addr and paste the output here (mask the MAC addresses like AA:BB:CC:xx:xx:xx if you want), I can tell you exactly which GATEWAY_IP to pick and which interfaces to use for UPSTREAM/DOWNSTREAM.

where does this comes from ?

[mac, ip, event.data.customer.email, plan, durationDays, ip, durationDays]

The array you are asking about

JavaScript

```
[mac, ip, event.data.customer.email, plan, durationDays, ip, durationDays]
```

comes from **this line** in the /paystack-webhook endpoint of the code I gave you:

JavaScript

```
await conn.execute(
  `INSERT INTO paid_devices (mac, ip, email, plan, expires_at, active)
   VALUES (?, ?, ?, ?, DATE_ADD(NOW(), INTERVAL ? DAY), 1)
   ON DUPLICATE KEY UPDATE
     ip = ?, expires_at = DATE_ADD(NOW(), INTERVAL ? DAY), active = 1`,
  [mac, ip, event.data.customer.email, plan, durationDays, ip, durationDays]
);
```

Where each value comes from (breakdown)

Position	Value	Where it comes from	Purpose in the query

1	mac	Detected earlier via getMacFromIp(ip) (ARP lookup)	Primary key – unique device identifier
2	ip	req.connection.remoteAddress from the webhook request	Current IP of the device (for logging / possible future use)
3	event.data.customer.email	Paystack webhook payload → event.data.customer.email	Email the user entered during payment
4	plan	From transaction metadata (sent when initializing payment)	Which subscription plan was purchased
5	durationDays	Calculated earlier from the plan (daily=1, weekly=7, monthly=30)	How many days to set for expires_at
6	ip (again)	Same as position 2	Used in ON DUPLICATE KEY UPDATE to refresh the IP
7	durationDays (again)	Same as position 5	Used in ON DUPLICATE KEY UPDATE to extend/refresh expiry

Why some values are repeated

The SQL query uses an **INSERT ... ON DUPLICATE KEY UPDATE** pattern.

- The first five ? placeholders are for the **INSERT** part (new record).
- The last two ? placeholders are for the **UPDATE** part (what to change if the MAC already exists).

That's why ip and durationDays appear twice in the array — once for insert, once for update.

Visual mapping

SQL

```
INSERT INTO paid_devices (mac, ip, email, plan, expires_at, active)

VALUES              ( ? , ? ,  ?  , ?  , DATE_ADD..., 1 )



ON DUPLICATE KEY UPDATE

   ip = ?, expires_at = DATE_ADD(NOW(), INTERVAL ? DAY), active = 1



Array positions:      1   2   3    4    5          6    7
```

So in short:

- The array is the ordered list of values that replace the ? placeholders in the prepared SQL statement.
- Order must match exactly how the placeholders appear in the query string.

If you ever change the columns/order in the SQL, you must also change the order in this array accordingly.

Let me know if you want to modify the query (e.g., add more fields, remove email, change expiry logic) — I can update both the SQL and the array for you.

<u>so, the mac address of the device is gotten from the ip ?</u>

Yes, **exactly** — in this setup, the MAC address of the connected device is obtained **from its IP address** using ARP (Address Resolution Protocol).

Here's how and why it works that way:

How it happens in the code

The function getMacFromIp(ip) does this:

JavaScript

```javascript
async function getMacFromIp(ip) {
  return new Promise((resolve) => {
    exec(`arp -i ${DOWNSTREAM_IFACE} -a | grep ${ip}`, (err, stdout) => {
      if (err || !stdout) return resolve(null);
      const match = stdout.match(/([0-9A-Fa-f]{2}[:-]){5}([0-9A-Fa-f]{2})/);
      resolve(match ? match[0].toUpperCase() : null);
    });
  });
}
```

What this command does:

Bash

```bash
arp -i wlan1 -a | grep 192.168.100.50
```

- arp -i wlan1 -a → shows the ARP table for the interface where your clients (hotspot users) are connected
- The ARP table is a cache maintained by the Linux kernel
- Every time a device on your local network (192.168.100.x) communicates, the kernel learns and stores: **IP address** → **MAC address** mapping
- So when we grep for a specific IP (the client's IP), we get back its MAC address from that table

Example real output of arp -a:

text

```text
? (192.168.100.50) at aa:bb:cc:dd:ee:ff [ether] on wlan1

? (192.168.100.51) at 11:22:33:44:55:66 [ether] on wlan1
```

→ We extract the MAC with a regex: aa:bb:cc:dd:ee:ff

Why we use MAC instead of just IP

- **IP can change** (DHCP lease expires, device reconnects → new IP)
- **MAC is hardware-based and (usually) fixed** for the device's WiFi interface → much more reliable for identifying the same device over time
- That's why we store mac as the primary key in the database — it survives IP changes, reboots, etc.

Important limitations & real-world notes

1. **ARP table entry must exist**
    - The device must have sent at least one packet (ARP request) so the kernel knows its MAC
    - Usually happens automatically when the device tries to reach the internet or DNS
    - If no entry yet → getMacFromIp returns null → payment page can show "Please try browsing a website first" or retry
2. **MAC randomization (privacy feature)**
    - Modern phones (iOS 14+, Android 10+) use **randomized MAC addresses** by default for each WiFi network
    - This means the same physical device may appear with a **different MAC** every time it reconnects
    - Workarounds (if this becomes a problem):
        - Ask users to disable MAC randomization for your SSID (in phone WiFi settings)
        - Or use **IP + user-agent + email** combination as a "soft" identifier instead
        - Or require login with phone/email after payment (more reliable than MAC alone)
3. **Alternative ways to get MAC (if ARP fails)**
    - Use **Nodogsplash** with ndsctl — it can provide client MAC directly in auth requests
    - Or parse /proc/net/arp directly in code instead of shell exec

In short: **Yes, we get the MAC from the IP** via the ARP cache — it's the standard and most reliable way in a Linux-based hotspot setup like yours.

If MAC randomization becomes an issue in testing, let me know — we can switch to email-based auth or add a simple login step after payment. Want to test the MAC detection part alone? I can give you a standalone script.

<u>is this MAC randomization constant and default on all random devices ?</u>

No, MAC address randomization is not constant (the same MAC every time) nor does it change every single connection on all devices — it depends on the operating system, version, device manufacturer, and user settings.

Here's the current situation (as of 2026) for major platforms — based on how modern smartphones and other devices behave by default:

Android (most devices running Android 10 and later)

- **Enabled by default** since Android 10 (2019) for privacy when connecting to Wi-Fi networks.
- **Type of randomization**:
    - **Persistent** (default on most devices): The randomized MAC is generated once per Wi-Fi network (SSID) and **stays the same** for every reconnection to that same network — until the user forgets the network, factory resets, or (on Android 12+) certain conditions trigger re-randomization (e.g., after 24 hours in some cases, or on reconnect if non-persistent mode is active).
    - **Non-persistent** (introduced in Android 12, used in specific cases): The MAC can change on every connection or after a period (more aggressive privacy, but less common as default).
- Most everyday Android phones use **persistent randomization** by default → same random MAC for your hotspot SSID across connections (good for your subscription tracking if users don't forget the network).
- Users can disable it per-network: Settings → Wi-Fi → gear icon → Privacy → "Use device MAC" (turns off randomization for that SSID only).

iOS / iPhone (iOS 14 and later)

- **Enabled by default** since iOS 14 (2020) — called "Private Wi-Fi Address" or "Private Address".
- Behavior: Generates a **random MAC per network** and **keeps it persistent** for that SSID (same random MAC on reconnects).
- Extra twist: iOS rotates (re-randomizes) the MAC **every 24 hours** after connecting, or immediately if the user forgets the network and rejoins.
- Users can disable per-network: Settings → Wi-Fi → tap (i) next to network → toggle "Private Wi-Fi Address" off → "Use device MAC".
- Very consistent: Same device usually keeps one random MAC for your SSID unless they forget/re-add it or wait 24+ hours.

Other Common Devices (2026)

- **Windows 10/11 laptops**: MAC randomization is **off by default**. Users must manually enable it (Settings → Network → Wi-Fi → Hardware properties → "Random hardware addresses").
- **macOS**: Off by default (no randomization unless manually enabled in advanced settings).
- **Linux laptops/desktops**: Usually **no randomization** by default (uses real MAC) unless tools like macchanger are used or specific distros enable it.
- **Older phones** (pre-Android 10 / pre-iOS 14): Almost never randomize — they use the real hardware MAC.

Summary: Is it constant or changing every time?

- **Not every time** on most modern smartphones: The default is **persistent per-network** — the random MAC stays the same when reconnecting to your hotspot SSID (great for your MySQL tracking using MAC as key).
- It only changes when:
  - User forgets the network and rejoins (new random MAC generated).
  - On iOS: Every ~24 hours automatically.
  - On some Android: If non-persistent mode triggers (less common by default).
  - Device is factory reset or network settings cleared.

What this means for your paid hotspot project

- **Most users** (Android/iOS modern defaults) will keep the **same random MAC** for your SSID across multiple connections → your database check will work reliably (once they pay, they stay authorized until expiry or they forget the network).
- **Edge cases** (rare but possible):
  - Users who disable randomization (use real MAC) → even better for tracking.
  - Users who frequently forget/rejoin → new MAC → treated as new unpaid user (they'll need to pay again).
- Recommendation: In your payment success page / instructions, add a note:
  "For best experience, keep this network saved on your device and don't forget it. This ensures your subscription stays active without re-payment."

If randomization causes too many issues in practice (e.g., high re-payment complaints), we can switch to **email + OTP login** after payment (more reliable, no MAC dependency).

Test it yourself: Connect a few phones to your hotspot, pay once, disconnect/reconnect multiple times, and check if the same MAC appears in arp -a output — it usually will.

Let me know if you want to add email-based fallback auth in the code!

I have bougth tplink acher ax55, but i have not bought starlink. but i have a modern wifi that i used to browse the internet before, can't i use it temporarily ? for the purpose of testing and working things out ?

Yes, **you can absolutely use your existing modem/router's WiFi temporarily** instead of Starlink for testing and development. In fact, that's the smartest way to start — you avoid waiting for Starlink delivery/setup, you can test everything indoors with stable power, and you can iron out all the code, database, payment flow, and portal issues before going live with Starlink.

Your current modem/router (the one you already use for browsing) will act as the **upstream internet source** (just like Starlink will later). The flow remains almost identical:

Temporary Testing Flow

- Existing modem/router → provides internet to your Kali laptop (connect Kali via Ethernet cable to the modem/router's LAN port, or wirelessly if needed)
- Kali laptop → acts as the gateway / captive portal controller (blocks internet, redirects to payment page, checks MySQL, unblocks on payment)
- Kali laptop → Ethernet cable → TP-Link Archer AX55 (in Access Point mode) (AX55 broadcasts the "MyPaidWiFi" SSID that test users connect to)

Once everything works perfectly with your current internet, you only need to swap the upstream connection to Starlink later — the code and configuration stay the same.

How to Set It Up Right Now (No Starlink Needed)

1. **Connect Hardware**
    - Modem/router LAN port → Ethernet cable → Kali laptop's Ethernet port (or use USB-Ethernet adapter if your laptop has no RJ45)
    - Kali laptop → another Ethernet cable → AX55 WAN/Internet port
2. **Put AX55 in Access Point Mode** (very important)
    - Temporarily connect to AX55 (default WiFi or Ethernet)
    - Go to http://tplinkwifi.net or 192.168.0.1
    - Login → Advanced → Operation Mode → **Access Point** → Save & Reboot
    - After reboot, set SSID to "MyPaidWiFi" (or whatever you want), set password
    - AX55 now acts only as a WiFi extender/AP — no routing/NAT/DHCP

On Kali – Find Interfaces & Set GATEWAY_IP

Run these commands:
Bash
```
ip link show        # list all interfaces

ip addr show        # see IPs and status
```

3. iwconfig # see WiFi interfaces (if using USB adapter)
   Typical results:
   - eth0 or enpXsY → wired connection to your modem/router (upstream = internet)
   - wlan1 or wlpXsY → USB WiFi adapter (downstream = hotspot side)

Example safe values for testing:
JavaScript
```
const DOWNSTREAM_IFACE = 'wlan1';     // USB WiFi adapter → connected to AX55

const UPSTREAM_IFACE   = 'eth0';      // wired to your current modem/router
```

4. const GATEWAY_IP = '192.168.200.1'; // different from your modem's subnet!
   Important: Choose a subnet that does **not** overlap with your modem/router's range.

Check your modem's subnet first:
Bash
```
ip route show default
```

   - # or look at your Kali IP when connected normally
     - If modem gives 192.168.1.x → don't use 192.168.1.1 for GATEWAY_IP
     - If modem gives 192.168.100.x → don't use 192.168.100.1
     - Safe choices: 192.168.200.1, 192.168.50.1, 10.20.30.1

Update dnsmasq.conf (if you're using it separately)
Bash
```
sudo nano /etc/dnsmasq.conf
```
Make sure:
text
```
interface=wlan1

dhcp-range=192.168.200.50,192.168.200.150,255.255.255.0,12h

dhcp-option=3,192.168.200.1
```

5. dhcp-option=6,8.8.8.8
6. **Run the Script & Test**
   - sudo node wifi_manager.js
   - Connect a test phone to "MyPaidWiFi" (AX55 SSID)
   - Open any website → should redirect to your payment page
   - Pay (use Paystack test keys/cards)
   - Check MySQL table → see if entry added
   - Device should now have internet

Advantages of Testing This Way

- Zero cost for internet during development

- Much faster feedback loop (no waiting for Starlink dish alignment, power issues, etc.)
- You can test in controlled environment (your room/house)
- When Starlink arrives: just unplug modem cable → plug Starlink Ethernet → done (same code)

Quick Checklist Before Running

- AX55 is in **Access Point** mode (not Router mode)
- Kali has internet from modem/router (ping 8.8.8.8 works)
- USB WiFi adapter shows * AP in iw list
- GATEWAY_IP subnet is different from modem/router subnet
- MySQL is running and table exists

If you run into interface/subnet confusion, paste output of these two commands and I'll tell you exactly what values to put:

Bash

```
ip addr show

ip route show
```

You're very close now — testing with your current internet is perfect. Let me know when you hit any error or need tweaks!

<u>Also, will paystack allow connection from localhost ?</u>

No, **Paystack does not allow webhook notifications to be sent directly to localhost** (or any 127.0.0.1, ::1, or internal/private IP addresses). This is explicitly stated in their official documentation and applies in both **test mode** and **live mode**.

Why localhost is blocked

- Webhooks are server-to-server HTTP POST requests sent from Paystack's servers to your URL.
- Paystack servers are on the public internet → they cannot reach your local machine (localhost) behind your NAT/firewall/router.
- Their docs repeatedly say:
  "Ensure your webhook URL is publicly available (localhost URLs can't receive events)"

This is a deliberate security/reliability design — no payment gateway allows localhost webhooks for the same reason.

How to test webhooks locally anyway

The standard and recommended way (used by almost every Paystack developer in Nigeria) is to expose your local server to the public internet temporarily using a tunneling tool. The most popular one is **ngrok** (free tier is sufficient for testing).

Quick steps to test on localhost with ngrok

1. **Download and install ngrok**
   - Go to https://ngrok.com/download
   - Sign up for free account (required for stable URLs)
   - Unzip and run in terminal (or add to PATH):
     Bash
     ```
     ./ngrok authtoken YOUR_AUTH_TOKEN   # from dashboard
     ```
2. **Start your Node.js server**
   - Make sure your webhook endpoint is listening (e.g., on port 3000 or 80):
     Bash
     ```
     sudo node wifi_manager.js   # or whatever port
     ```
3. **Expose localhost with ngrok**
   Bash
   ```
   ngrok http 3000   # or your port, e.g., ngrok http 80
   ```
   - You'll see something like:
     text
     ```
     Forwarding   https://abcd-1234.ngrok-free.app → http://localhost:3000
     ```
   - Copy the **https** URL (e.g., https://abcd-1234.ngrok-free.app)
4. **Add this URL to Paystack dashboard**
   - Log in to https://dashboard.paystack.com
   - Settings → API Keys & Webhooks
   - Scroll to **API Configuration – Test Mode**
   - Paste the ngrok URL + your webhook path into **Test Webhook URL**:
     text
     ```
     https://abcd-1234.ngrok-free.app/paystack-webhook
     ```
   - Save
5. **Test it**
   - Make a test payment using Paystack test cards (e.g., 408 408 408 408 408 1, CVV 408, expiry any future date)
   - Or click **Send Test Webhook** in dashboard to simulate
   - Watch your Node.js terminal — webhook should hit, log the event, update DB, unblock device
   - Ngrok dashboard shows incoming requests for debugging

Important Notes

- **Free ngrok** gives random URLs each time you restart — re-paste into Paystack dashboard every session.
- Paid ngrok ($5–20/month) gives static URLs (e.g., custom subdomain) — better for longer dev.
- Alternatives to ngrok: localtunnel, Cloudflare Tunnel, Bore — but ngrok is the easiest and most documented for Paystack in Nigeria.
- In **production** (with Starlink live): Deploy your Node.js server to a public host (e.g., Render.com free tier, Vercel, Railway, or VPS like DigitalOcean ~₦5k–₦15k/month) → use that real URL in Paystack live webhook field.

For testing your current setup (with existing modem + AX55), this is perfect — no Starlink needed yet.

When you're ready for Starlink swap, just change the upstream connection — webhook flow stays identical.

Let me know if you need help installing ngrok, securing the webhook (verify signature), or testing the first payment!

Oh, can all these works on a web hosting ?

I have a namecheap web hosting. I thought it can only work on local environment on my pc only?

Yes, **parts of your setup can work on Namecheap shared hosting**, but **not the full thing** — and definitely **not the core captive portal / traffic control** part that makes the paid WiFi hotspot function.

Let me explain clearly what can and cannot run on Namecheap shared hosting (based on their current features in 2026), so you know exactly what to expect.

What CAN work on Namecheap shared hosting

Namecheap **does support Node.js** on shared hosting plans (Stellar, Stellar Plus, Stellar Business, etc.) via their "Setup Node.js App" tool in cPanel.

- **Your Node.js payment portal server** (Express app with payment page, Paystack integration, MySQL checks, webhook handler) — **yes, this can run there**.
    - You can upload your code, select Node.js version (they support up to 24.x as of now), set application root, run npm install, and start it.

    - It will be publicly accessible via your domain (e.g., https://yourdomain.com/payment).
    - Paystack webhooks will work perfectly (they send POST requests to your public URL — no localhost issue once hosted).
- **MySQL database** — yes, Namecheap provides MariaDB/MySQL in cPanel → create your paid_devices table there.
- **Simple payment success/failure pages** — yes.

So the **frontend payment flow + backend logic + database** can live on Namecheap shared hosting.

What CANNOT work on Namecheap shared hosting

The **critical hotspot enforcement part** (captive portal / blocking/unblocking users) **cannot run there** — and this is the main reason the full system needs your local PC or a dedicated server.

- **iptables / nftables rules** — shared hosting does **not** give you root access or firewall control. You cannot run iptables -A FORWARD ... or iptables -t nat -A PREROUTING ... to block/redirect user traffic. This is only possible on a machine where you control the network stack (your Kali laptop, Raspberry Pi, VPS with root, etc.).
- **hostapd / dnsmasq / Nodogsplash** — these require root-level network interface control to create a WiFi access point, hand out IPs, and handle captive redirection. Shared hosting is just web hosting — no ability to create/manage WiFi interfaces or act as a router/gateway.
- **ARP scanning / MAC detection in real-time for connected devices** — this requires low-level access to the network interface (e.g., arp -i wlan1 -a), which shared hosting doesn't expose.

In short:

Namecheap shared hosting is great for **hosting a normal website or API** (your payment backend), but it **cannot** act as the network gateway/router that enforces the "no internet until paid" rule. That part **must run locally** on a machine that controls the WiFi network (your Kali laptop + USB adapter + AX55 in AP mode).

Best Hybrid Approach for You Right Now

Since you already have the AX55 and are testing without Starlink:

1. **Run the full enforcement logic locally on Kali** (as before) — use your existing modem/router as upstream internet for testing.
2. **Move only the payment portal + webhook + MySQL to Namecheap** later (for 24/7 availability without keeping your laptop on).
    - Host Node.js Express on Namecheap (via cPanel → Setup Node.js App).

- Use a public URL like https://yourdomain.com/paystack-webhook in Paystack dashboard.
- In your local Kali code:
    - Redirect users to the Namecheap-hosted payment page instead of localhost.
    - After payment, Paystack webhook hits Namecheap → updates MySQL there.
    - To unblock: Either:
        - Poll the DB from Kali every few seconds (simple).
        - Or use a lightweight API endpoint on Namecheap that Kali calls to confirm payment.

This way you get:

- Full control locally for testing/hotspot enforcement.
- Always-on payment backend on Namecheap (no need to keep laptop running 24/7 once live).

If you want true 24/7 hotspot without your PC always on, you'll eventually need a small always-on device (Raspberry Pi or cheap VPS with root access), but for now your current modem + Kali + AX55 is perfect for development.

Let me know if you want help:

- Moving the Express server to Namecheap (upload steps, cPanel config).
- Changing the redirect URL in code to point to your hosted domain.
- Or setting up polling/webhook callback between local Kali and hosted server.

You're making great progress — testing locally first is exactly right!

<u>So, had it been i have vps server, everything would work on vps, including the kali part ?</u>

Yes — if you had a **VPS server** (with root access), **most of the system could run entirely on the VPS**, including the equivalent of the "Kali part" (the gateway, captive portal enforcement, traffic blocking/unblocking, MAC/IP detection, etc.).

But there is one very important limitation: **the VPS cannot create or broadcast a real Wi-Fi hotspot** (SSID that phones/laptops connect to wirelessly). That part still needs a physical device with Wi-Fi hardware.

What would work 100% on a VPS

- **All the software logic** you wrote or we discussed:
  - Node.js + Express payment portal
  - Paystack integration (initiate payment + webhook handler)
  - MySQL/MariaDB database (paid_devices table)
  - Webhook processing & payment success logic
  - API to check payment status / authorize devices
- **Captive portal enforcement logic** (the "Kali part"):
  - iptables / nftables rules to block/redirect traffic
  - ARP scanning or client detection
  - Nodogsplash / openNDS with external FAS authentication to your Node.js server
  - MAC/IP-based authorization
  - Unblock on payment success
- **Routing / NAT / DHCP** (dnsmasq or systemd-networkd)

→ In other words: **everything except physically creating the Wi-Fi access point**.

What still needs a physical device (cannot run on VPS)

- **Wi-Fi radio / access point creation** A VPS is a virtual machine in a data center — it has **no Wi-Fi hardware** and no direct access to physical network interfaces that can operate in AP (Access Point) mode. You cannot run hostapd on a VPS to broadcast an SSID that phones can see and connect to.

How the full system would look with a VPS

Typical production setup using VPS + physical AP:

1. **VPS** (e.g., DigitalOcean, Linode, Vultr, Contabo — ~~₦5k–₦20k~~/month for 2–4 GB RAM instance)
   - Runs:
     - Node.js + payment backend + webhook
     - MySQL database
     - Nodogsplash / openNDS in bridge mode (or just the FAS auth server)
     - iptables rules (if the VPS is the gateway)
   - Has a **public IP** → Paystack webhooks go here directly (no ngrok needed)

Physical device at your location (one of these options)

2. Option A – Recommended for simplicity & range
   - TP-Link Archer AX55 (or better router) flashed with **OpenWrt**
   - OpenWrt connects to VPS as its authentication backend
   - AX55 broadcasts "MyPaidWiFi" → users connect here
   - Traffic forwarded to VPS → VPS decides block/allow

3. Option B – Keep using Kali laptop
    - Kali connects to VPS via VPN or API
    - Kali acts as local gateway + hotspot
    - VPS handles payment + DB
    - Kali polls VPS or VPS pushes updates
4. Option C – Cheapest long-term
    - Raspberry Pi 4/5 running OpenWrt or Raspberry Pi OS
    - Pi broadcasts Wi-Fi + runs lightweight openNDS
    - Forwards auth requests to VPS Node.js

Summary: VPS + Physical AP is the ideal production setup

Component	Can run fully on VPS?	Needs physical hardware?	Recommendation for you now
Payment page + Paystack	Yes	No	Move this to Namecheap or VPS
MySQL database	Yes	No	Yes, on VPS or Namecheap
Webhook handler	Yes	No	VPS (public IP)
Captive portal logic / auth	Yes	No	VPS
iptables blocking/unblocking	Yes (on VPS gateway)	No	VPS
Wi-Fi hotspot broadcast (SSID)	No	Yes	AX55 in AP mode or Pi
DHCP / IP assignment	Yes	No	VPS or physical AP

Short answer:

Yes — with a VPS you can run **almost everything** (including the "Kali-like" enforcement logic), but you **still need a physical device** (your AX55 in AP mode, a Raspberry Pi, or an OpenWrt router) to actually broadcast the Wi-Fi network that users connect to.

For now (testing phase), keep using your current modem + Kali laptop + AX55 — it's free and perfect for development.

When you're ready for 24/7 live operation without your laptop always on, get a small VPS (~₦8,000–₦15,000/month) + keep the AX55 as the AP.

Want me to recommend specific cheap VPS providers in Nigeria (with good latency from Lagos) or help plan the migration from local Kali to VPS + AX55?